MkSafeNet_Kids - Complete Project Documentation

Version: 1.0.0
Last Updated: June 9, 2026
Format: All-in-One Complete Guide
Purpose: Comprehensive documentation for developers, DevOps, and project stakeholders

---

📋 Table of Contents

---

1. Project Overview

1.1 About MkSafeNet_Kids

MkSafeNet_Kids is a comprehensive web-based educational platform designed to teach children and students about online safety and digital citizenship through interactive chat-based scenarios and real-world safety education.

The application guides students through digital safety scenarios where they engage in chat-based conversations, make decisions in realistic situations, and learn about online safety consequences in a supportive learning environment. Teachers can create and manage learning sessions, and admins oversee the entire system.

Target Users:

- **Students**: Participate in interactive safety scenarios
- **Teachers**: Create sessions, track student progress, manage courses
- **Admins**: Manage schools, teachers, scenarios, and view system statistics

1.2 Key Features

- **Interactive Chat Scenarios** – Students respond to real-world online safety situations
- **Role-Based Access Control** – Admin, Teacher, and Student roles with tailored experiences
- **Session Management** – Teachers create and monitor student learning sessions with QR code access
- **Certificate Generation** – Automated certificate PDF generation upon course completion
- **JWT-based Authentication** – Secure token-based authentication for all users
- **School & Organization Management** – Admins manage schools and teacher accounts
- **Real-time Feedback** – Immediate feedback on student responses with consequence education
- **Progress Tracking** – Score tracking, scenario results, and badge/certificate rewards

1.3 Technology Stack

Layer	Technology	Version
Backend Runtime	Java	17+
Backend Framework	Spring Boot	3.x

Layer	Technology	Version
Backend Security	Spring Security	-
Backend ORM	Hibernate + Spring Data JPA	-
Database	SQLite	-
Frontend Framework	Vue	3.x
Frontend Build Tool	Vite	5.x
State Management	Pinia	2.x
HTTP Client	Axios	1.x
Frontend Router	Vue Router	4.x
Authentication	JWT (JSON Web Tokens)	-
Build Tool (Backend)	Maven	3.8+
Package Manager (Frontend)	npm	Latest

1.4 Project Structure

```
MkSafeNet_Kids/
├── backend/                                # Spring Boot REST API
│   ├── src/main/java/com/mksafenet/
│   │   ├── MksafenetApplication.java       # Spring Boot entry point
│   │   ├── controller/                     # REST endpoints (6 controllers)
│   │   │   ├── AuthController.java
│   │   │   ├── ChatController.java
│   │   │   ├── CertificateController.java
│   │   │   ├── TeacherController.java
│   │   │   ├── ScenarioController.java
│   │   │   └── AdminController.java
│   │   ├── service/                        # Business logic layer
│   │   ├── model/                          # JPA entities
│   │   ├── repository/                     # Data access layer
│   │   ├── dto/                            # Data Transfer Objects (8 DTOs)
│   │   ├── config/                         # Configuration classes
│   │   │   ├── SecurityConfig.java
│   │   │   ├── JwtAuthFilter.java
│   │   │   └── DataSeeder.java
│   │   ├── converter/                      # JSON converters
│   │   └── util/                           # Utilities (JwtUtil)
│   ├── src/main/resources/
│   │   ├── application.properties          # Configuration
│   │   └── templates/                      # HTML templates
│   ├── pom.xml                             # Maven dependencies
│   └── target/                             # Build output
│
├── frontend/                               # Vue 3 + Vite frontend
│   ├── src/
```

```
│   │   ├── main.js                          # Entry point
│   │   ├── App.vue                          # Root component
│   │   ├── api/                             # Axios configuration
│   │   │   └── index.js
│   │   ├── stores/                          # Pinia stores
│   │   │   └── auth.js
│   │   ├── views/                           # Page components
│   │   │   ├── LoginView.vue
│   │   │   ├── ChatView.vue
│   │   │   ├── admin/
│   │   │   │   └── AdminDashboard.vue
│   │   │   └── teacher/
│   │   │       └── TeacherDashboard.vue
│   │   ├── components/                      # Reusable components
│   │   │   └── ConsequenceModal.vue
│   │   ├── router/                          # Vue Router config
│   │   │   └── index.js
│   │   └── assets/                          # Static assets
│   ├── index.html                           # HTML template
│   ├── package.json                         # npm dependencies
│   ├── vite.config.js                       # Vite configuration
│   └── dist/                                # Build output
│
├── docs/                                    # Documentation (separate files)
├── mksafenet.db                             # SQLite database
├── README.md                                # Project overview
├── CONTRIBUTING.md                          # Contribution guidelines
├── CHANGELOG.md                             # Version history
├── start-backend.bat                        # Windows startup script
├── start-frontend.bat                       # Windows startup script
└── .git/                                    # Git repository
```

---

2. Getting Started

2.1 Quick Start

Backend Quick Start

```
cd backend
mvn clean install
mvn spring-boot:run
# Backend runs on http://localhost:8080
```

Frontend Quick Start

```
cd frontend
npm install
npm run dev
# Frontend runs on http://localhost:5173
```

Using Startup Scripts

```
# Terminal 1
.\start-backend.bat

# Terminal 2
.\start-frontend.bat
```

2.2 Prerequisites

System Requirements

For Backend:

- **Java**: 17 or higher (Download)
- **Maven**: 3.8+ (Download)

Verify installation:

```
java -version
mvn -version
```

For Frontend:

- **Node.js**: 18 or higher (Download)
- **npm**: Comes with Node.js

Verify installation:

```
node --version
npm --version
```

For Git (Recommended):

- **Git**: Latest version (Download)

Optional Tools

- **Docker**: For containerized deployment

- **SQLite Browser**: For database inspection
- **Postman**: For API testing
- **IDE**: IntelliJ IDEA, VS Code, or WebStorm

2.3 Local Development Setup

Step 1: Clone Repository

```
git clone https://github.com/yourorg/mksafenet.git
cd MkSafeNet_Kids
```

Step 2: Backend Configuration

Navigate to backend directory:

```
cd backend
```

Edit `src/main/resources/application.properties`:

```
# Database (SQLite)
spring.datasource.url=jdbc:sqlite:mksafenet.db
spring.datasource.driver-class-name=org.sqlite.JDBC

# Hibernate
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false

# JWT Configuration
jwt.secret=mksafenet-super-secret-jwt-key-change-in-production-32chars
jwt.expiration=86400000

# Frontend URL (CORS)
app.frontend.url=http://localhost:5173

# Server Port
server.port=8080
```

Step 3: Build Backend

```
mvn clean install
```

Step 4: Run Backend

```
mvn spring-boot:run
```

Expected output:

```
...
Started MksafenetApplication in 5.123s
Tomcat started on port(s): 8080
```

Step 5: Frontend Setup

Navigate to frontend directory:

```
cd ..\frontend
```

Install dependencies:

```
npm install
```

Run development server:

```
npm run dev
```

Expected output:

```
VITE v5.0.12 ready in 123 ms

➜ Local: http://localhost:5173/
```

2.4 Accessing the Application

Login

1. Open browser: `http://localhost:5173`
2. Login with default credentials:
    - Username: `admin`
    - Password: `admin`

Default Users

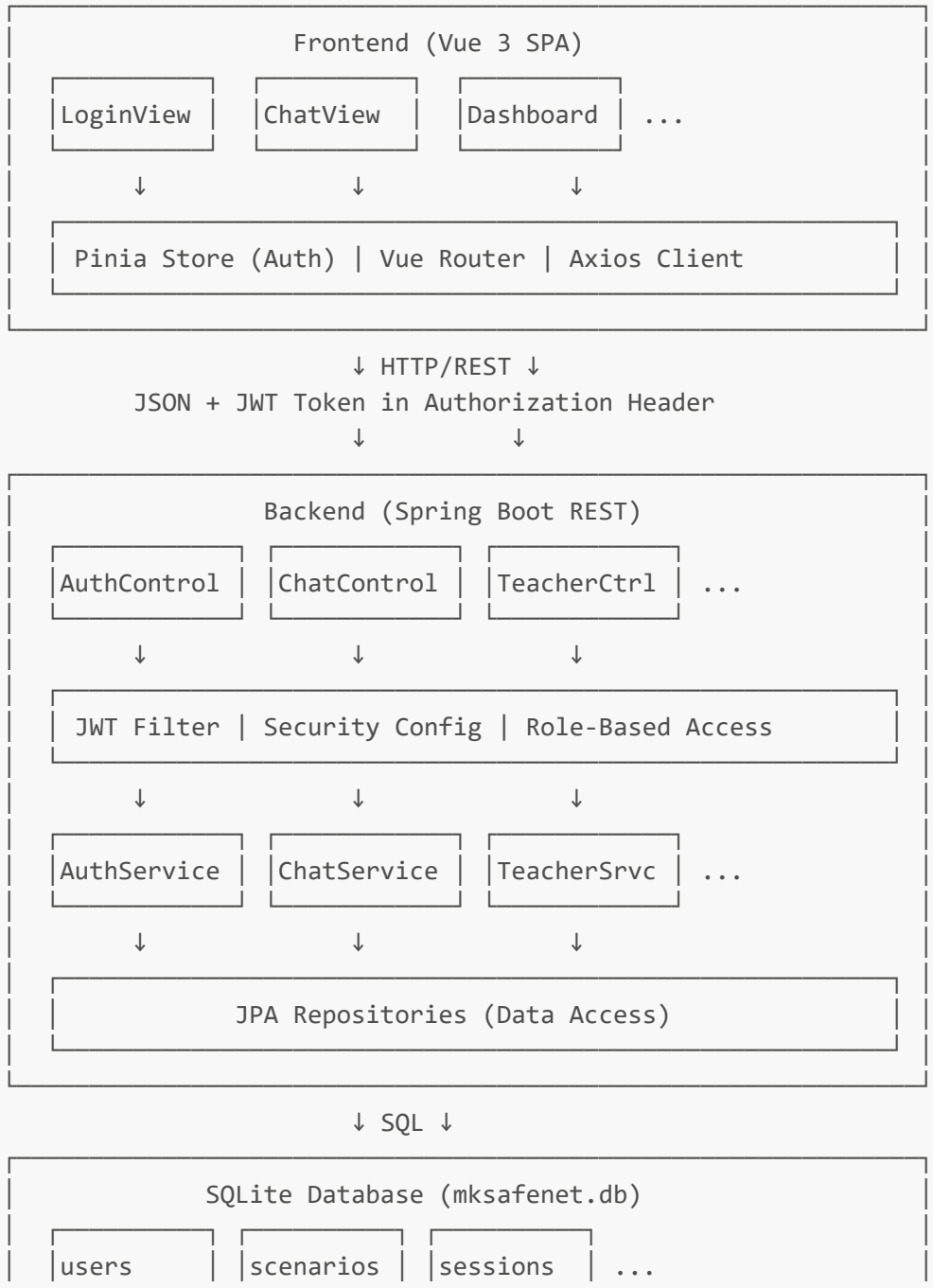
Created by `DataSeeder` on first run:

Username	Password	Role	School
admin	admin	ADMIN	N/A
teacher1	teacher1	TEACHER	Test School
teacher2	teacher2	TEACHER	Test School

3. System Architecture

3.1 Architecture Overview

MkSafeNet_Kids follows a **client-server architecture** with clear separation of concerns:



```
│  └──────────┘  └──────────┘  └──────────┘                    │
└──────────────────────────────────────────────────────────────┘
```

3.2 Backend Architecture

Controllers (REST Endpoints)

Location: `backend/src/main/java/com/mksafenet/controller/`

Controller	Base Path	Purpose	Roles
`AuthController`	`/api/auth`	User authentication	PUBLIC
`ChatController`	`/api/chat`	Chat scenarios	STUDENT
`CertificateController`	`/api/certificates`	PDF certificates	STUDENT
`TeacherController`	`/api/teacher`	Session management	TEACHER
`ScenarioController`	`/api/admin/scenarios`	Scenario CRUD	ADMIN
`AdminController`	`/api/admin`	System management	ADMIN

Service Layer

Location: `backend/src/main/java/com/mksafenet/service/`

- `AuthService` – Login and authentication
- `ChatService` – Chat sessions and scenarios
- `CertificateService` – PDF generation
- `TeacherService` – Session lifecycle
- `ScenarioService` – Scenario management
- `AdminService` – Admin operations

Model/Entity Layer

JPA entities in `backend/src/main/java/com/mksafenet/model/`

- `User` – Users (teachers, students, admins)
- `School` – Educational institutions
- `Session` – Learning sessions
- `StudentSession` – Student participation
- `Scenario` – Safety scenarios
- `ChatHistory` – Chat records

DTO Layer

Location: `backend/src/main/java/com/mksafenet/dto/`

Request DTOs:

- `LoginRequest` – Login credentials

- `ChatStartRequest` – Start chat
- `ChatRespondRequest` – Submit answer

Response DTOs:

- `LoginResponse` – Auth response
- `ChatResponseDto` – Chat/scenario response
- `ChatMessageDto` – Individual message
- `ScenarioOptionDto` – Answer option
- `ScenarioResultDto` – Result tracking

Repository Layer

Spring Data JPA repositories for database queries

- `UserRepository` – User queries
- `SchoolRepository` – School queries
- `SessionRepository` – Session queries
- `ScenarioRepository` – Scenario queries

Configuration & Security

Location: `backend/src/main/java/com/mksafenet/config/`

- `SecurityConfig` – Spring Security configuration, CORS, filter chain
- `JwtAuthFilter` – JWT token validation
- `DataSeeder` – Sample data initialization

3.3 Frontend Architecture

File Structure

```
frontend/src/
├── main.js                    # Entry point
├── App.vue                    # Root component
├── index.html                 # HTML template
├── api/
│   └── index.js               # Axios instance with interceptors
├── stores/
│   └── auth.js                # Pinia authentication store
├── router/
│   └── index.js               # Vue Router configuration
├── views/
│   ├── LoginView.vue          # Login page
│   ├── ChatView.vue           # Chat interface
│   ├── admin/
│   │   └── AdminDashboard.vue
│   └── teacher/
│       └── TeacherDashboard.vue
├── components/
│   └── ConsequenceModal.vue   # Reusable modal
```

```
└── assets/
    └── logo.png              # Static files
```

State Management (Pinia)

Store in `frontend/src/stores/auth.js`:

```
{
  token,                // JWT token
  role,                 // User role
  displayName,          // User name
  schoolId,             // School ID
  schoolName,           // School name
  isLoggedIn,            // Computed property
  login(),              // Action
  logout()              // Action
}
```

Routing (Vue Router)

Routes configured in `frontend/src/router/index.js`:

- `/login` – Login page
- `/dashboard` – Role-based dashboard
- `/chat` – Chat interface
- `/admin/*` – Admin pages (ADMIN role)
- `/teacher/*` – Teacher pages (TEACHER role)

API Client (Axios)

Configured in `frontend/src/api/index.js`:

- Base URL: `/api`
- JWT token attached to all requests
- 401 redirect to login
- Error handling

3.4 Data Flow Examples

Login Flow

```
User enters credentials
    ↓
POST /api/auth/login
    ↓
AuthController.login()
    ↓
AuthService.login() validates and generates JWT
```

```
    ↓
Return token + user info
    ↓
Frontend stores token in localStorage
    ↓
Pinia store updated
    ↓
Axios interceptor includes token in future requests
    ↓
Redirect to dashboard
```

Chat Scenario Flow

```
Student scans QR code or enters token
    ↓
POST /api/chat/start
    ↓
ChatService validates session and loads first scenario
    ↓
Return first question with options
    ↓
Frontend renders chat and buttons
    ↓
Student selects answer
    ↓
POST /api/chat/respond
    ↓
ChatService evaluates answer (correct/incorrect)
    ↓
If incorrect: show consequence messages
If correct: show success and next scenario
    ↓
Continue or complete quiz
    ↓
Final score and certificate
```

3.5 Deployment Architecture

Development

```
Localhost
  ├─ http://localhost:8080  (Backend)
  ├─ http://localhost:5173  (Frontend)
  └─ mksafenet.db          (Local SQLite)
```

Production (Docker)

```
Docker Containers
  ├─ Backend Container (Java)
  │   └─ Port 8080 (internal)
  ├─ Frontend Container (Nginx)
  │   └─ Port 80 (internal)
  └─ Persistent Volume
      └─ mksafenet.db
```

Production (VPS)

```
Linux Server (systemd)
  ├─ Backend Service
  │   ├─ java -jar backend.jar
  │   └─ Port 8080 (localhost)
  ├─ Nginx Reverse Proxy
  │   ├─ HTTPS (port 443)
  │   ├─ Routes /api → backend:8080
  │   └─ Serves frontend static files
  └─ SQLite Database
      └─ /opt/mksafenet/mksafenet.db
```

---

4. API Reference

4.1 Authentication

Base URL

```
http://localhost:8080/api
```

POST `/api/auth/login`

Authenticates user and returns JWT token.

Request:

```
{
  "username": "admin",
  "password": "admin"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "role": "ADMIN",
  "username": "admin",
  "displayName": "Administrator",
  "schoolId": null,
  "schoolName": null
}
```

Error (401 Unauthorized):

```
{
  "error": "Invalid username or password"
}
```

curl Example:

```
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin"}'
```

4.2 Chat API

GET `/api/chat/session/{token}`

Validates session token and returns session details.

Parameters:

- `token` (path) – Session token from QR code

Response (200 OK):

```
{
  "valid": true,
  "sessionName": "Period 1 Safety Class",
  "schoolName": "Test School"
}
```

Error (404 Not Found):

```
{
  "error": "Session not found or inactive"
```

```
}
```

POST /api/chat/start

Starts a new chat session for a student.

Request:

```
{
  "sessionToken": "abc123xyz789",
  "studentName": "John Doe"
}
```

Response (200 OK):

```
{
  "studentId": "john-doe-001",
  "phase": "INTRO",
  "messages": [
    {
      "type": "bot",
      "text": "Welcome to Online Safety Challenge!",
      "delayMs": 500,
      "icon": "👋"
    }
  ],
  "scenarioId": null,
  "question": null,
  "options": null,
  "score": null,
  "grade": null,
  "passed": null,
  "correctCount": null,
  "totalScenarios": null,
  "badges": null,
  "scenarioResults": null
}
```

POST /api/chat/respond

Submits student answer to scenario.

Request:

```
{
  "studentId": "john-doe-001",
```

```
  "answer": "A"
}
```

Response (200 OK) - Correct Answer:

```
{
  "studentId": "john-doe-001",
  "phase": "SCENARIO",
  "correct": true,
  "scenarioId": 2,
  "question": "What do you do?",
  "options": [
    {"key": "A", "text": "Option A"},
    {"key": "B", "text": "Option B"}
  ],
  "messages": [
    {
      "type": "success",
      "text": "Great choice!",
      "delayMs": 500,
      "icon": "✅"
    }
  ],
  "correctCount": 1,
  "totalScenarios": 3
}
```

Response (200 OK) - Incorrect with Consequence:

```
{
  "studentId": "john-doe-001",
  "phase": "CONSEQUENCE",
  "correct": false,
  "consequenceType": "ACCOUNT_HACKED",
  "consequenceMessages": [
    {
      "type": "consequence",
      "text": "Your account was hacked!",
      "delayMs": 800,
      "icon": "🔓"
    }
  ],
  "messages": [...]
}
```

Response (200 OK) - Quiz Complete:

```
{
  "studentId": "john-doe-001",
  "phase": "COMPLETE",
  "score": 85,
  "grade": "A",
  "passed": true,
  "correctCount": 8,
  "totalScenarios": 10,
  "badges": ["Safety Conscious", "Quick Learner"],
  "scenarioResults": [
    {
      "scenarioId": 1,
      "scenarioTitle": "Stranger Contact",
      "selectedAnswer": "A",
      "correct": true,
      "pointsEarned": 10
    }
  ]
}
```

4.3 Certificate API

GET `/api/certificates/download?name=<name>`

Downloads certificate PDF for student.

Parameters:

- `name` (query) – Student's full name

Response:

- Content-Type: `application/pdf`
- File: `certificate-<name>.pdf`

curl Example:

```
curl -X GET "http://localhost:8080/api/certificates/download?name=John%20Doe" \
  -H "Authorization: Bearer <token>" \
  -o certificate.pdf
```

4.4 Teacher API

All require `TEACHER` role.

POST `/api/teacher/sessions`

Creates new learning session.

Request:

```
{
  "name": "Period 1 - Safety Class"
}
```

Response (200 OK):

```
{
  "id": 1,
  "name": "Period 1 - Safety Class",
  "token": "abc123xyz789",
  "active": true,
  "createdAt": "2024-01-15T10:30:00Z",
  "studentCount": 0,
  "teacherName": "Ms. Smith"
}
```

GET /api/teacher/sessions

Retrieves all sessions for teacher.

Response (200 OK):

```
[
  {
    "id": 1,
    "name": "Period 1 - Safety Class",
    "token": "abc123xyz789",
    "active": true,
    "createdAt": "2024-01-15T10:30:00Z",
    "studentCount": 5,
    "teacherName": "Ms. Smith"
  }
]
```

GET /api/teacher/sessions/{id}

Gets session details including student results.

Response (200 OK):

```
{
  "id": 1,
  "name": "Period 1 - Safety Class",
  "token": "abc123xyz789",
```

```
  "active": true,
  "students": [
    {
      "studentName": "John Doe",
      "score": 85,
      "passed": true,
      "completedAt": "2024-01-15T11:15:00Z"
    }
  ]
}
```

GET `/api/teacher/sessions/{id}/qr`

Generates QR code for session.

Response:

- Content-Type: `image/png`
- QR code image

PUT `/api/teacher/sessions/{id}/toggle`

Activates or deactivates session.

Request:

```
{
  "active": false
}
```

Response (200 OK):

```
{
  "success": true
}
```

4.5 Scenario API (Admin)

All require `ADMIN` role.

GET `/api/admin/scenarios`

Lists all scenarios.

Response (200 OK):

```
[
  {
    "id": 1,
    "title": "Stranger Contact",
    "description": "Someone you don't know...",
    "question": "What should you do?",
    "options": [
      {"key": "A", "text": "Block and report"},
      {"key": "B", "text": "Reply and continue"}
    ],
    "correctAnswer": "A"
  }
]
```

GET /api/admin/scenarios/{id}

Gets scenario details.

POST /api/admin/scenarios

Creates new scenario.

Request:

```
{
  "title": "New Scenario",
  "description": "Description",
  "question": "Question?",
  "options": [
    {"key": "A", "text": "Option A"},
    {"key": "B", "text": "Option B"}
  ],
  "correctAnswer": "A",
  "consequence": "Consequence message"
}
```

PUT /api/admin/scenarios/{id}

Updates scenario.

DELETE /api/admin/scenarios/{id}

Deletes scenario.

Response (200 OK):

```
{
  "message": "Scenario deleted successfully"
```

```
}
```

4.6 Admin API

All require `ADMIN` role.

GET `/api/admin/schools`

Lists all schools.

POST `/api/admin/schools`

Creates new school.

Request:

```
{
  "name": "New School",
  "address": "456 Oak Ave",
  "city": "Shelbyville"
}
```

POST `/api/admin/teachers`

Creates new teacher.

Request:

```
{
  "username": "newteacher",
  "password": "securepass123",
  "displayName": "Mr. New Teacher",
  "schoolId": 1
}
```

GET `/api/admin/teachers`

Lists all teachers.

GET `/api/admin/stats`

Gets system statistics.

Response (200 OK):

```
{
  "totalSchools": 1,
```

```
  "totalTeachers": 2,
  "totalStudentSessions": 15,
  "averageScore": 78.5,
  "totalCertificatesIssued": 10
}
```

4.7 DTOs Reference

DTO	Purpose	Fields
`LoginRequest`	Login credentials	`username`, `password`
`LoginResponse`	Auth response	`token`, `role`, `username`, `displayName`, `schoolId`, `schoolName`
`ChatStartRequest`	Start chat	`sessionToken`, `studentName`
`ChatRespondRequest`	Submit answer	`studentId`, `answer`
`ChatResponseDto`	Chat response	`studentId`, `phase`, `messages`, `scenarioId`, `question`, `options`, `correct`, `score`, `grade`, `passed`, `correctCount`, `totalScenarios`, `badges`, `scenarioResults`
`ChatMessageDto`	Single message	`type`, `text`, `delayMs`, `icon`
`ScenarioOptionDto`	Answer option	`key`, `text`
`ScenarioResultDto`	Result	`scenarioId`, `scenarioTitle`, `selectedAnswer`, `correct`, `pointsEarned`

4.8 Response Codes

Code	Description
`200`	Success
`400`	Bad Request (validation error)
`401`	Unauthorized (missing/invalid token)
`403`	Forbidden (insufficient permissions)
`404`	Not Found (resource doesn't exist)
`500`	Internal Server Error

5. Database Schema

5.1 Database Overview

- **Type**: SQLite (file-based)
- **File**: `mksafenet.db`
- **ORM**: Hibernate + Spring Data JPA
- **DDL Auto**: `update` (automatic schema management)

5.2 Entity Relationships

```
USERS (1) ←── (1) SCHOOLS
  │
  ├── (1) ←──── (N) SESSIONS
  │                  │
  │                  └─ (1) ←──── (N) STUDENT_SESSIONS
  │                                    │
  │                                    └─ (N) ←──── (1) SCENARIOS
  │                                    │
  │                                    └─ (N) ←──── (1) CHAT_HISTORY
  │
  └── (1) ←──── (N) [User to User relationships]
```

5.3 Tables Documentation

USERS Table

```sql
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  display_name VARCHAR(255) NOT NULL,
  role VARCHAR(50) NOT NULL,
  school_id INTEGER,
  created_at TIMESTAMP NOT NULL,
  FOREIGN KEY(school_id) REFERENCES schools(id)
);
```

Columns:

- `id` – Auto-increment primary key
- `username` – Unique login username
- `password` – Hashed password (bcrypt)
- `display_name` – User's full name
- `role` – ADMIN, TEACHER, or STUDENT
- `school_id` – Associated school (nullable for ADMIN)
- `created_at` – Account creation date

Default Data:

- admin / admin (ADMIN)
- teacher1 / teacher1 (TEACHER)
- teacher2 / teacher2 (TEACHER)

SCHOOLS Table

```
CREATE TABLE schools (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name VARCHAR(255) NOT NULL,
  address VARCHAR(255) NOT NULL,
  city VARCHAR(255) NOT NULL,
  created_at TIMESTAMP NOT NULL
);
```

Columns:

- `id` – Auto-increment primary key
- `name` – School name
- `address` – Street address
- `city` – City
- `created_at` – Creation date

SESSIONS Table

```
CREATE TABLE sessions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name VARCHAR(255) NOT NULL,
  token VARCHAR(255) NOT NULL UNIQUE,
  active BOOLEAN DEFAULT true,
  teacher_id INTEGER NOT NULL,
  created_at TIMESTAMP NOT NULL,
  FOREIGN KEY(teacher_id) REFERENCES users(id)
);
```

Columns:

- `id` – Auto-increment primary key
- `name` – Session name (e.g., "Period 1")
- `token` – Unique access token (used in QR code)
- `active` – Session active/inactive
- `teacher_id` – Session creator
- `created_at` – Creation date

STUDENT_SESSIONS Table

```
CREATE TABLE student_sessions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  student_name VARCHAR(255) NOT NULL,
  session_id INTEGER NOT NULL,
  score INTEGER,
  passed BOOLEAN,
  completed_at TIMESTAMP,
  FOREIGN KEY(session_id) REFERENCES sessions(id)
);
```

Columns:

- `id` – Auto-increment primary key
- `student_name` – Student's name
- `session_id` – Associated session
- `score` – Final score (0-100)
- `passed` – Whether passed
- `completed_at` – Completion timestamp

SCENARIOS Table

```
CREATE TABLE scenarios (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  question TEXT NOT NULL,
  options TEXT NOT NULL,
  correct_answer VARCHAR(10) NOT NULL,
  consequence TEXT,
  created_at TIMESTAMP NOT NULL
);
```

Columns:

- `id` – Auto-increment primary key
- `title` – Scenario title
- `description` – Setup/context
- `question` – Student question
- `options` – JSON array of options
- `correct_answer` – Correct option key
- `consequence` – Consequence message if wrong
- `created_at` – Creation date

Example options (JSON):

```
[
  {"key":"A","text":"Block and report"},
  {"key":"B","text":"Reply and continue"}
]
```

CHAT_HISTORY Table

```
CREATE TABLE chat_history (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  student_session_id INTEGER NOT NULL,
  scenario_id INTEGER NOT NULL,
  student_answer VARCHAR(10) NOT NULL,
  is_correct BOOLEAN NOT NULL,
  points_earned INTEGER DEFAULT 0,
  created_at TIMESTAMP NOT NULL,
  FOREIGN KEY(student_session_id) REFERENCES student_sessions(id),
  FOREIGN KEY(scenario_id) REFERENCES scenarios(id)
);
```

Columns:

- `id` – Auto-increment primary key
- `student_session_id` – Student's session
- `scenario_id` – Scenario question
- `student_answer` – Answer given (e.g., "A")
- `is_correct` – Correct/incorrect
- `points_earned` – Points awarded
- `created_at` – Answer timestamp

5.4 Schema Management

Auto-Migration

Spring Boot automatically creates/updates schema on startup via Hibernate `ddl-auto=update`

Adding a New Table

1. **Create Entity:**

```
@Entity
@Table(name = "new_table")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class NewEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    @Column(nullable = false)
    private String name;

    @PrePersist
    public void prePersist() {
        createdAt = LocalDateTime.now();
    }
}
```

2. **Create Repository:**

```
@Repository
public interface NewEntityRepository extends JpaRepository<NewEntity, Long> {
    Optional<NewEntity> findByName(String name);
}
```

3. **Restart Backend:**

```
mvn spring-boot:run
```

Hibernate automatically creates the table.

Querying Database

Using SQLite CLI:

```
sqlite3 backend/mksafenet.db

# List tables
.tables

# Show schema
.schema users

# Query data
SELECT * FROM users;

# Exit
.quit
```

Backups

Manual backup:

```
Copy-Item backend/mksafenet.db backend/mksafenet.db.backup
```

Production backup (cron job):

```
0 2 * * * tar -czf /backups/mksafenet-$(date +\%Y\%m\%d).tar.gz
/opt/mksafenet/mksafenet.db
```

---

6. Backend Development

6.1 Project Structure

```
backend/src/main/java/com/mksafenet/
├── controller/          # REST endpoints
│   ├── AuthController.java
│   ├── ChatController.java
│   ├── CertificateController.java
│   ├── TeacherController.java
│   ├── ScenarioController.java
│   └── AdminController.java
├── service/             # Business logic
│   ├── AuthService.java
│   ├── ChatService.java
│   ├── CertificateService.java
│   ├── TeacherService.java
│   ├── ScenarioService.java
│   └── AdminService.java
├── model/               # JPA entities
│   ├── User.java
│   ├── School.java
│   ├── Session.java
│   └── Scenario.java
├── repository/          # Data access
│   ├── UserRepository.java
│   ├── SchoolRepository.java
│   ├── SessionRepository.java
│   └── ScenarioRepository.java
├── dto/                 # Request/Response
│   ├── LoginRequest.java
│   ├── LoginResponse.java
│   ├── ChatStartRequest.java
│   └── [other DTOs]
├── config/              # Configuration
│   ├── SecurityConfig.java
│   ├── JwtAuthFilter.java
│   └── DataSeeder.java
├── converter/           # Type converters
```

```
├── util/                # Utilities
│   └── JwtUtil.java
└── MksafenetApplication.java
```

6.2 Adding Endpoints

Step 1: Create DTOs

Request DTO:

```java
package com.mksafenet.dto;

import lombok.Data;

@Data
public class MyRequestDto {
    private String field1;
    private Integer field2;
}
```

Response DTO:

```java
package com.mksafenet.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class MyResponseDto {
    private Long id;
    private String status;
    private String message;
}
```

Step 2: Create Service

```java
package com.mksafenet.service;

import com.mksafenet.dto.MyRequestDto;
import com.mksafenet.dto.MyResponseDto;
import com.mksafenet.model.MyEntity;
import com.mksafenet.repository.MyRepository;
```

```
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class MyService {

    private final MyRepository myRepository;

    public MyResponseDto handleRequest(MyRequestDto request) {
        // Validate input
        if (request.getField1() == null || request.getField1().isBlank()) {
            throw new IllegalArgumentException("field1 is required");
        }

        // Business logic
        MyEntity entity = new MyEntity();
        entity.setField1(request.getField1());
        entity.setField2(request.getField2());

        // Persist
        MyEntity saved = myRepository.save(entity);

        // Return response
        return MyResponseDto.builder()
            .id(saved.getId())
            .status("success")
            .message("Request processed successfully")
            .build();
    }
}
```

Step 3: Create Controller

```
package com.mksafenet.controller;

import com.mksafenet.dto.MyRequestDto;
import com.mksafenet.dto.MyResponseDto;
import com.mksafenet.service.MyService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/my")
@RequiredArgsConstructor
public class MyController {
```

```
    private final MyService myService;

    @PostMapping("/endpoint")
    @PreAuthorize("hasRole('TEACHER')")  // Role restriction
    public ResponseEntity<?> myEndpoint(@RequestBody MyRequestDto request) {
        try {
            MyResponseDto response = myService.handleRequest(request);
            return ResponseEntity.ok(response);
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest()
                .body(Map.of("error", e.getMessage()));
        } catch (Exception e) {
            return ResponseEntity.status(500)
                .body(Map.of("error", "Internal server error"));
        }
    }
}
```

Step 4: Update API Reference

Document new endpoint in API Reference section with examples.

6.3 Adding Entities

Create Entity Class

```
package com.mksafenet.model;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;

@Entity
@Table(name = "my_entity")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class MyEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 255)
    private String field1;

    @Column(nullable = true)
    private Integer field2;
```

```
    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(nullable = false, updatable = false)
    @Temporal(TemporalType.TIMESTAMP)
    private LocalDateTime createdAt;

    @PrePersist
    public void prePersist() {
        createdAt = LocalDateTime.now();
    }
}
```

Create Repository

```
package com.mksafenet.repository;

import com.mksafenet.model.MyEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface MyRepository extends JpaRepository<MyEntity, Long> {

    // Auto-generated query methods
    Optional<MyEntity> findByField1(String field1);
    List<MyEntity> findByField2(Integer field2);

    // Custom JPQL query
    @Query("SELECT m FROM MyEntity m WHERE m.field1 = :field1")
    List<MyEntity> findCustom(@Param("field1") String field1);
}
```

6.4 Using Transactions

```
@Service
@RequiredArgsConstructor
public class MyService {
    private final MyRepository myRepository;

    @Transactional  // Wraps method in transaction
    public void updateMultiple() {
        // All database operations here
```

```
        // Auto-rollback if exception occurs
        myRepository.save(entity1);
        myRepository.save(entity2);
        // If any save fails, all are rolled back
    }
}
```

6.5 Testing

Unit Test Example

```
package com.mksafenet.service;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

@DisplayName("MyService Tests")
class MyServiceTest {

    private MyService myService;

    @Mock
    private MyRepository myRepository;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
        myService = new MyService(myRepository);
    }

    @Test
    @DisplayName("Should handle request successfully")
    void testHandleRequest_Success() {
        // Arrange
        MyRequestDto request = new MyRequestDto();
        request.setField1("test");

        // Act
        MyResponseDto response = myService.handleRequest(request);

        // Assert
        assertNotNull(response);
        assertEquals("success", response.getStatus());
    }
```

```
    @Test
    @DisplayName("Should throw exception on invalid input")
    void testHandleRequest_InvalidInput() {
        // Arrange
        MyRequestDto request = new MyRequestDto();
        request.setField1(null);

        // Act & Assert
        assertThrows(IllegalArgumentException.class, () -> {
            myService.handleRequest(request);
        });
    }
}
```

Run Tests

```
cd backend
mvn -q test
```

6.6 Best Practices

1. **Separation of Concerns**
    - Controllers handle HTTP requests/responses
    - Services handle business logic
    - Repositories handle data access
    - DTOs for API contracts
2. **Error Handling**
    - Use exceptions for errors
    - Controllers catch and convert to responses
    - Avoid returning null
3. **Validation**
    - Validate input in services
    - Throw `IllegalArgumentException` for validation errors
    - Use `@Valid` on controller parameters
4. **Security**
    - Use `@PreAuthorize` for role-based access
    - Don't expose sensitive data
    - Hash passwords
5. **Naming**
    - Classes: PascalCase

- Methods/Fields: camelCase
- Database columns: snake_case
- Endpoints: kebab-case

6. **Documentation**

- Add Javadoc to public methods
- Document complex logic
- Update API reference

---

7. Frontend Development

7.1 Project Structure

```
frontend/src/
├── main.js                    # Entry point
├── App.vue                    # Root component
├── api/
│   └── index.js               # Axios configuration
├── stores/
│   └── auth.js                # Pinia store
├── router/
│   └── index.js               # Vue Router config
├── views/
│   ├── LoginView.vue
│   ├── ChatView.vue
│   ├── admin/
│   │   └── AdminDashboard.vue
│   └── teacher/
│       └── TeacherDashboard.vue
├── components/
│   └── ConsequenceModal.vue
└── assets/
    └── logo.png
```

7.2 Creating Views

File: `frontend/src/views/MyView.vue`

```
<template>
  <div class="my-view">
    <h1>My View</h1>

    <div v-if="loading" class="loading">Loading...</div>
    <div v-else-if="error" class="error">{{ error }}</div>
    <div v-else class="content">
      <p>{{ message }}</p>
```

```
      <button @click="fetchData">Refresh Data</button>
    </div>
  </div>
</template>

<script setup>
import { ref, onMounted } from 'vue'
import api from '../api/index.js'

const message = ref('Hello from MyView')
const loading = ref(false)
const error = ref(null)

const fetchData = async () => {
  loading.value = true
  error.value = null
  try {
    const res = await api.get('/my-endpoint')
    message.value = res.data.message
  } catch (err) {
    error.value = err.response?.data?.error || 'An error occurred'
  } finally {
    loading.value = false
  }
}

onMounted(() => {
  fetchData()
})
</script>

<style scoped>
.my-view {
  padding: 20px;
}

.loading, .error {
  padding: 10px;
  margin: 10px 0;
}

.error {
  color: red;
  background: #ffe0e0;
}

button {
  padding: 10px 20px;
  background: #007bff;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
```

```
button:hover {
  background: #0056b3;
}
</style>
```

7.3 Creating Components

File: `frontend/src/components/MyComponent.vue`

```
<template>
  <div class="component">
    <h2>{{ title }}</h2>
    <button @click="$emit('action')">{{ buttonText }}</button>
  </div>
</template>

<script setup>
defineProps({
  title: { type: String, required: true },
  buttonText: { type: String, default: 'Click Me' }
})

defineEmits(['action'])
</script>

<style scoped>
.component {
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

button {
  padding: 10px 20px;
  background: #28a745;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

button:hover {
  background: #218838;
}
</style>
```

7.4 API Integration

Making Requests:

```
// GET request
const res = await api.get('/endpoint')

// POST request
const res = await api.post('/endpoint', {
  field1: 'value1',
  field2: 'value2'
})

// PUT request
const res = await api.put('/endpoint/1', data)

// DELETE request
const res = await api.delete('/endpoint/1')
```

Error Handling:

```
<script setup>
const handleRequest = async () => {
  try {
    const response = await api.post('/endpoint', data)
    // Success
  } catch (err) {
    if (err.response?.status === 401) {
      // Unauthorized - interceptor handles redirect
    } else if (err.response?.status === 403) {
      // Forbidden
      alert('You do not have permission')
    } else if (err.response?.status === 400) {
      // Validation error
      alert(err.response.data.error)
    } else {
      // Server error
      alert('An error occurred')
    }
  }
}
</script>
```

7.5 State Management

Using Auth Store:

```
<script setup>
import { useAuthStore } from '../stores/auth.js'

const authStore = useAuthStore()
```

```
// Access state
const username = authStore.displayName
const role = authStore.role

// Call action
const logout = () => authStore.logout()
</script>
```

Creating a Store:

```
// frontend/src/stores/mystore.js
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import api from '../api/index.js'

export const useMyStore = defineStore('mystore', () => {
  const items = ref([])
  const loading = ref(false)

  const itemCount = computed(() => items.value.length)

  async function fetchItems() {
    loading.value = true
    try {
      const res = await api.get('/my-items')
      items.value = res.data
    } finally {
      loading.value = false
    }
  }

  function clearItems() {
    items.value = []
  }

  return { items, loading, itemCount, fetchItems, clearItems }
})
```

7.6 Best Practices

1. **Keep Components Small**
    - One responsibility per component
    - Extract reusable logic
2. **Reactive State**
    - Use `ref()` for primitives
    - Use Pinia for shared state

3. **Async Operations**
    - Use `async/await`
    - Handle errors with try/catch
    - Show loading states
4. **Performance**
    - Use `v-show` for frequent toggling
    - Lazy-load routes for large apps
    - Avoid inline functions
5. **Code Organization**
    - Separate template, script, style
    - Use descriptive names
    - Comment complex logic
6. **Styling**
    - Use scoped styles
    - Avoid inline styles
    - Consider CSS framework (Bootstrap, Tailwind)

---

8. Security & Authentication

8.1 Authentication Flow

Login Process

```
User submits credentials (username, password)
    ↓
POST /api/auth/login
    ↓
AuthService validates credentials (password comparison)
    ↓
If invalid: return 401 error
    ↓
If valid: JwtUtil.generateToken(user)
    ↓
Return JWT token + user info
    ↓
Frontend: localStorage.setItem('token', token)
    ↓
Frontend: Store in Pinia auth store
    ↓
Axios interceptor includes token in Authorization header
```

Token Validation

```
Client sends request with Authorization header
    ↓
Authorization: Bearer <token>
    ↓
JwtAuthFilter intercepts request
    ↓
Extract token from header
    ↓
JwtUtil.isTokenValid(token)
    ↓
If invalid: return 401 Unauthorized
    ↓
If valid: Extract username from token claims
    ↓
Load user from database
    ↓
Create SecurityContext with user
    ↓
Request proceeds with user context
```

8.2 JWT Configuration

Backend Configuration

File: `backend/src/main/resources/application.properties`

```
# JWT Secret (change to strong random value in production)
jwt.secret=mksafenet-super-secret-jwt-key-change-in-production-32chars

# JWT Expiration (milliseconds)
# 86400000 = 24 hours
jwt.expiration=86400000
```

Generating Secure Secret

```
# Generate random 32-character secret
$bytes = [System.Security.Cryptography.RandomNumberGenerator]::GetBytes(32)
[Convert]::ToBase64String($bytes)
```

Or use online generator and Base64 encode.

Requirements:

- Minimum 32 characters

- Mix of uppercase, lowercase, numbers, special characters
- Different value for production

Environment Variables

For production, use environment variables instead of hardcoding:

```
jwt.secret=${JWT_SECRET}
jwt.expiration=${JWT_EXPIRATION:86400000}
```

Set variables before running:

```
$env:JWT_SECRET = "your-strong-secret-here"
java -jar app.jar
```

8.3 Role-Based Access Control

Three Roles

Role	Access	Endpoints
ADMIN	System administrator	`/api/admin/*`, `/api/admin/scenarios/*`
TEACHER	Educator, session management	`/api/teacher/*`
STUDENT	Participant	`/api/chat/*`, `/api/certificates/*`

Backend Protection

```
@RestController
@RequestMapping("/api/admin/scenarios")
@PreAuthorize("hasRole('ADMIN')")  // Entire controller protected
@RequiredArgsConstructor
public class ScenarioController {

    @PostMapping
    public ResponseEntity<?> createScenario(@RequestBody Scenario scenario) {
        // Only ADMIN can access
    }

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")  // Method-level override
    public ResponseEntity<?> getAllScenarios() {
        // Protected
    }
}
```

Frontend Protection

```
// frontend/src/router/index.js
const routes = [
  {
    path: '/admin',
    component: AdminDashboard,
    meta: { requiresAuth: true, roles: ['ADMIN'] }
  },
  {
    path: '/teacher',
    component: TeacherDashboard,
    meta: { requiresAuth: true, roles: ['TEACHER'] }
  }
]

router.beforeEach((to, from, next) => {
  const authStore = useAuthStore()

  if (to.meta.requiresAuth && !authStore.isLoggedIn) {
    next('/login')
  } else if (to.meta.roles && !to.meta.roles.includes(authStore.role)) {
    next('/unauthorized')
  } else {
    next()
  }
})
```

8.4 Security Best Practices

1. Password Hashing

```
// Use Spring Security PasswordEncoder
@Configuration
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

// In AuthService
String hashedPassword = passwordEncoder.encode(rawPassword);
```

2. HTTPS in Production

- Always use HTTPS (not HTTP)
- Obtain SSL certificate (Let's Encrypt)

- Configure CORS for HTTPS domain

3. Secure Token Storage

Frontend:

```
// Store in localStorage (vulnerable to XSS)
localStorage.setItem('token', token)

// Better: HttpOnly cookies (not accessible to JavaScript)
// Backend sets HttpOnly cookie automatically
// Frontend sends cookie with requests
```

4. Token Refresh

For long sessions, implement refresh token:

```
POST /api/auth/refresh
Body: { refreshToken: "..." }
Response: { accessToken: "...", refreshToken: "..." }
```

5. CORS Configuration

```
@Configuration
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.cors(cors -> cors.configurationSource(request -> {
            CorsConfiguration config = new CorsConfiguration();
            config.setAllowedOrigins(List.of(
                "http://localhost:5173",      // Dev
                "https://yourdomain.com"      // Prod
            ));
            config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
            config.setAllowedHeaders(List.of("*"));
            config.setAllowCredentials(true);
            return config;
        }));
        // ... rest of config
    }
}
```

6. Input Validation

```
@PostMapping("/create")
public ResponseEntity<?> create(@RequestBody @Valid CreateRequest request) {
    // @Valid triggers bean validation
}
```

7. Rate Limiting

Implement rate limiting to prevent brute-force attacks (optional but recommended):

```
// Consider adding Spring Cloud Gateway or custom interceptor
@Component
public class RateLimitFilter {
    private final RateLimiter rateLimiter = RateLimiter.create(100);
}
```

8. Audit Logging

```
@Service
public class AuditService {
    public void log(String action, User user, String details) {
        logger.info("USER_ACTION: {} by {} - {}",
            action, user.getUsername(), details);
    }
}
```

---

9. Deployment

9.1 Deployment Options

Option	Complexity	Cost	Setup Time
Docker	Medium	Low	30 min
VPS + systemd	Low	Low	20 min
Heroku	Low	Medium	15 min
AWS/Azure	High	Medium-High	1-2 hours

9.2 Docker Deployment

Backend Dockerfile

File: `backend/Dockerfile`

```
FROM openjdk:17-slim

WORKDIR /app

ARG JAR_FILE=target/mksafenet-*.jar

COPY ${JAR_FILE} app.jar

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
  CMD java -cp app.jar org.springframework.boot.loader.JarLauncher

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Frontend Dockerfile

File: `frontend/Dockerfile`

```
FROM node:18-alpine as builder

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# Production stage
FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Docker Compose

File: `docker-compose.yml`

```
version: '3.8'

services:
  backend:
    build:
```

```
      context: ./backend
      dockerfile: Dockerfile
    container_name: mksafenet-backend
    ports:
      - "8080:8080"
    environment:
      DATABASE_URL: jdbc:sqlite:/data/mksafenet.db
      JWT_SECRET: ${JWT_SECRET:-change-me-in-production}
      FRONTEND_URL: http://localhost:3000
      SPRING_JPA_HIBERNATE_DDL_AUTO: update
    volumes:
      - db-data:/data
    restart: unless-stopped

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: mksafenet-frontend
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: unless-stopped

volumes:
  db-data:
```

Build and Run

```
# Build backend JAR
cd backend
mvn clean package -DskipTests

cd ..

# Build and run containers
docker-compose up --build -d

# Stop containers
docker-compose down

# View logs
docker-compose logs -f backend
docker-compose logs -f frontend
```

9.3 Linux VPS Deployment

Prerequisites

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y openjdk-17-jre-headless maven
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
```

Backend Setup

Create systemd service:

```
sudo mkdir -p /opt/mksafenet
sudo chown $USER:$USER /opt/mksafenet

# Build JAR
cd backend
mvn clean package -DskipTests
cp target/mksafenet-*.jar /opt/mksafenet/backend.jar

# Create service file
sudo tee /etc/systemd/system/mksafenet-backend.service > /dev/null <<EOF
[Unit]
Description=MkSafeNet Backend
After=network.target

[Service]
User=mksafenet
WorkingDirectory=/opt/mksafenet
EnvironmentFile=/opt/mksafenet/.env
ExecStart=/usr/bin/java -Xmx512m -jar backend.jar
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

# Create user
sudo useradd -r -s /bin/bash mksafenet 2>/dev/null || true
sudo chown mksafenet:mksafenet /opt/mksafenet

# Enable and start
sudo systemctl enable mksafenet-backend
sudo systemctl start mksafenet-backend
sudo systemctl status mksafenet-backend
```

Environment file:

File: `/opt/mksafenet/.env`

```
JWT_SECRET=your-strong-secret-key-here
APP_FRONTEND_URL=https://yourdomain.com
DATABASE_URL=jdbc:sqlite:/opt/mksafenet/mksafenet.db
```

Frontend Setup

```
# Build frontend
cd frontend
npm install
npm run build

# Copy to web root
sudo mkdir -p /var/www/mksafenet
sudo cp -r dist/* /var/www/mksafenet/
```

9.4 Nginx Configuration

File: `/etc/nginx/sites-available/mksafenet`

```
# HTTP to HTTPS redirect
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;
    return 301 https://$server_name$request_uri;
}

# HTTPS server
server {
    listen 443 ssl http2;
    server_name yourdomain.com www.yourdomain.com;

    # SSL certificates (Let's Encrypt)
    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    root /var/www/mksafenet;
    index index.html;

    # API proxy
    location /api/ {
        proxy_pass http://localhost:8080/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }

    # SPA fallback
```

```
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Static assets caching
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

Enable and Start Nginx

```
sudo ln -s /etc/nginx/sites-available/mksafenet /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl enable nginx
sudo systemctl start nginx
```

SSL Certificate

```
# Install Certbot
sudo apt install -y certbot python3-certbot-nginx

# Obtain certificate
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com

# Auto-renewal
sudo systemctl enable certbot.timer
sudo systemctl start certbot.timer
```

9.5 Production Checklist

- [ ] Change JWT secret to strong value
- [ ] Set `app.frontend.url` to production domain
- [ ] Enable HTTPS with valid certificate
- [ ] Configure database (SQLite or external)
- [ ] Set up automated backups
- [ ] Configure monitoring and logging
- [ ] Test all endpoints in production
- [ ] Enable rate limiting
- [ ] Configure CORS for production domain
- [ ] Set up firewall (allow only ports 80/443)
- [ ] Update dependencies for security patches
- [ ] Test database backups and restores
- [ ] Monitor performance and errors

- [ ] Set up alerting for critical issues

---

10. Testing

10.1 Testing Strategy

Test Type	Scope	Tools	Coverage
Unit	Individual methods	JUnit 5, Mockito	≥80%
Integration	API + Database	Spring Boot Test	≥70%
E2E	Complete workflows	Playwright	Key flows
Manual	UI/UX	Browser	Edge cases

10.2 Backend Testing

Unit Test Example

```
package com.mksafenet.service;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

@DisplayName("AuthService Tests")
class AuthServiceTest {

    private AuthService authService;

    @Mock
    private UserRepository userRepository;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
        authService = new AuthService(userRepository, jwtUtil);
    }

    @Test
    @DisplayName("Should login successfully with valid credentials")
    void testLogin_ValidCredentials() {
        // Arrange
        LoginRequest request = new LoginRequest();
```

```
        request.setUsername("admin");
        request.setPassword("admin");

        User user = User.builder()
            .username("admin")
            .password("$2a$10$...") // bcrypt hash
            .role(Role.ADMIN)
            .build();

        when(userRepository.findByUsername("admin"))
            .thenReturn(Optional.of(user));

        // Act
        LoginResponse response = authService.login(request);

        // Assert
        assertNotNull(response);
        assertEquals("admin", response.getUsername());
        verify(userRepository).findByUsername("admin");
    }
}
```

Run Backend Tests

```
cd backend
mvn test
mvn clean test jacoco:report  # With coverage
```

10.3 Frontend Testing

Unit Test Example

```
import { describe, it, expect, beforeEach, vi } from 'vitest'
import { setActivePinia, createPinia } from 'pinia'
import { useAuthStore } from '../auth'
import api from '../../api/index'

vi.mock('../../api/index')

describe('Auth Store', () => {
  beforeEach(() => {
    setActivePinia(createPinia())
  })

  it('should initialize with empty token', () => {
    const authStore = useAuthStore()
    expect(authStore.token).toBeNull()
    expect(authStore.isLoggedIn).toBe(false)
  })
```

```
  it('should store token after login', async () => {
    const authStore = useAuthStore()

    api.post.mockResolvedValue({
      data: {
        token: 'test-token',
        role: 'ADMIN'
      }
    })

    await authStore.login('admin', 'admin')

    expect(authStore.token).toBe('test-token')
    expect(authStore.isLoggedIn).toBe(true)
  })
})
```

Run Frontend Tests

```
cd frontend
npm install --save-dev vitest @vue/test-utils jsdom
npm test
npm test -- --coverage
```

10.4 Manual Testing

Test Checklist

Authentication:

- [ ] Login with valid credentials
- [ ] Login with invalid credentials
- [ ] Logout functionality
- [ ] Token persisted in localStorage
- [ ] Automatic redirect on 401

Teacher Features:

- [ ] Create new session
- [ ] View all sessions
- [ ] Generate QR code
- [ ] Toggle session active/inactive
- [ ] View student results

Student Features:

- [ ] Join session via token
- [ ] View scenario questions

- [ ] Select answer options
- [ ] View immediate feedback
- [ ] Progress through scenarios
- [ ] View final score
- [ ] Download certificate

Admin Features:

- [ ] View all schools
- [ ] Create new school
- [ ] Create new teacher
- [ ] Create/edit scenarios
- [ ] View statistics

UI/UX:

- [ ] Responsive on mobile/tablet/desktop
- [ ] No console errors
- [ ] Loading spinners appear
- [ ] Error messages clear
- [ ] Navigation works
- [ ] Buttons are clickable

---

11. Troubleshooting

11.1 Backend Issues

Build Failures

Error: "Maven not found"

```
# Install Maven
# Download from https://maven.apache.org/
# Add to PATH environment variable
```

Error: "Port 8080 is already in use"

```
# Find process
netstat -ano | findstr :8080

# Kill process (replace PID with actual)
taskkill /PID 1234 /F

# Or change port in application.properties
# server.port=8081
```

Error: "Cannot find symbol"

```
cd backend
mvn clean install -U
```

Runtime Errors

Error: "401 Unauthorized"

- Verify token in localStorage
- Check Authorization header included
- Verify JWT secret matches
- Re-login if token expired

Error: "CORS error"

- Check `app.frontend.url` in application.properties
- Ensure it matches frontend URL exactly
- Include protocol, domain, and port

Error: "Database is locked"

```
# Kill Java process
taskkill /IM java.exe /F

# Delete database
Remove-Item backend/mksafenet.db -Force

# Restart backend
mvn spring-boot:run
```

11.2 Frontend Issues

Build Failures

Error: "npm not found"

```
# Install Node.js from https://nodejs.org/
# Includes npm
```

Error: "Module not found"

```
cd frontend
Remove-Item node_modules -Recurse -Force
```

```
npm install
```

Error: "Unexpected token"

- Check for missing brackets: `}`, `]`, `)`
- Check for mismatched quotes
- Check Vue template syntax

Runtime Errors

Error: "Cannot read property 'X' of undefined"

```
<!-- Use optional chaining -->
{{ user?.name }}

<!-- Or conditional rendering -->
<div v-if="user">{{ user.name }}</div>
```

Error: "Cannot connect to server"

- Verify backend is running on port 8080
- Check browser console for exact error
- Verify CORS configuration

Error: "White screen of death"

1. Open Developer Console (F12)
2. Check for JavaScript errors
3. Check Network tab for failed requests
4. Look for missing imports or syntax errors

11.3 Common Errors

Error	Solution
"Invalid username or password"	Use correct credentials (default: admin/admin)
"Session not found"	Create new session and get fresh token
"Forbidden" (403)	Check user role matches endpoint
"Resource not found" (404)	Check URL spelling and API reference
"Internal server error" (500)	Check backend logs for details
"Cannot connect to server"	Start backend on port 8080
"Failed to load module"	Run `npm install` to get dependencies
"Port already in use"	Kill existing process or change port

---

12. Contributing & Maintenance

12.1 Contribution Guidelines

Branching Strategy

- **Feature**: `feature/<short-description>`
- **Fix**: `fix/<short-description>`
- **Branch from**: `main` or `develop`

Pull Request Process

1. Create feature branch
2. Make changes and commit
3. Push branch
4. Open Pull Request
5. Wait for code review
6. Address feedback
7. Merge when approved

Commit Conventions

Use present-tense, imperative style:

- ✅ Good: "Add login endpoint"
- ❌ Bad: "Added login endpoint"

Examples

```
git checkout -b feature/new-scenario-type
# Make changes
git add .
git commit -m "Add new scenario type support"
git push origin feature/new-scenario-type
# Open PR on GitHub
```

12.2 Code Standards

Java Code Style

- Use Spring Boot conventions
- Use Lombok annotations
- Keep methods focused
- Add Javadoc for public methods

Example:

```
/**
 * Validates user credentials and generates JWT token.
 *
 * @param request Login request with username and password
 * @return LoginResponse with token and user info
 * @throws BadCredentialsException if credentials are invalid
 */
public LoginResponse login(LoginRequest request) {
    // Implementation
}
```

JavaScript/Vue Code Style

- Use ES modules
- Use `const`/`let` (never `var`)
- Follow existing project style
- Use Prettier for formatting

Example:

```
// ✅ Good
const result = await api.get('/endpoint')

// ❌ Bad
var result = api.get('/endpoint')
```

Database Naming

- Tables: `snake_case`
- Columns: `snake_case`
- Foreign keys: `<table>_id`

Example:

```
CREATE TABLE user_sessions (
  id INTEGER PRIMARY KEY,
  user_id INTEGER NOT NULL,
  session_name VARCHAR(255),
  created_at TIMESTAMP NOT NULL,
  FOREIGN KEY(user_id) REFERENCES users(id)
);
```

12.3 Release Process

Step 1: Update Version

Backend (`backend/pom.xml`):

```
<version>1.1.0</version>
```

Frontend (`frontend/package.json`):

```
{
  "version": "1.1.0"
}
```

Step 2: Update Changelog

```
## [1.1.0] - 2026-07-01

### Added
- New feature description
- Another feature

### Fixed
- Bug fix description
```

Step 3: Commit and Tag

```
git add CHANGELOG.md backend/pom.xml frontend/package.json
git commit -m "Bump version to 1.1.0"
git tag -a v1.1.0 -m "Release version 1.1.0"
git push origin v1.1.0
```

Step 4: Create Release

Create GitHub release with version and changelog.

12.4 Changelog

Format

```
## [X.Y.Z] - YYYY-MM-DD

### Added
- New features

### Changed
- Modified features
```

```
### Fixed
- Bug fixes

### Removed
- Removed features

### Deprecated
- Deprecated features

### Security
- Security fixes
```

Semantic Versioning

- **MAJOR** (X.0.0): Breaking changes
- **MINOR** (x.Y.0): New features
- **PATCH** (x.y.Z): Bug fixes

Examples:

- 1.0.0 – First release
- 1.1.0 – New features added
- 1.1.1 – Bug fix
- 2.0.0 – Breaking changes

---

13. Quick Reference

13.1 Common Commands

Backend

```
# Build
cd backend
mvn clean install

# Run
mvn spring-boot:run

# Test
mvn test

# Coverage report
mvn clean test jacoco:report

# Build JAR only
mvn clean package -DskipTests
```

Frontend

```
cd frontend

# Install dependencies
npm install

# Run dev server
npm run dev

# Build for production
npm run build

# Preview production build
npm run preview

# Run tests
npm test

# Format code
npm run format
```

Docker

```
# Build and run
docker-compose up --build -d

# Stop
docker-compose down

# Logs
docker-compose logs -f backend
docker-compose logs -f frontend

# Clean up
docker-compose down -v
```

Git

```
# Clone repository
git clone https://github.com/yourorg/mksafenet.git

# Create feature branch
git checkout -b feature/your-feature

# Commit changes
git add .
git commit -m "Your commit message"
```

```
# Push to remote
git push origin feature/your-feature

# Create pull request
# (via GitHub website)

# Merge to main
# (after PR approval)
```

SQLite

```
# Open database
sqlite3 backend/mksafenet.db

# List tables
.tables

# Show schema
.schema users

# Query data
SELECT * FROM users;

# Exit
.quit
```

13.2 Configuration Files

Backend Configuration

File: `backend/src/main/resources/application.properties`

Key settings:

- `spring.datasource.url` – Database connection
- `spring.jpa.hibernate.ddl-auto` – Schema management (update, create, none)
- `jwt.secret` – JWT signing key
- `jwt.expiration` – Token expiration in milliseconds
- `app.frontend.url` – Frontend URL for CORS
- `server.port` – Server port (default 8080)

Frontend Configuration

File: `frontend/vite.config.js`

```
export default {
  server: {
```

```
    port: 5173,
    host: 'localhost'
  }
}
```

File: `frontend/.env.production`

```
VITE_API_BASE_URL=https://api.yourdomain.com
VITE_APP_TITLE=MkSafeNet Kids
```

Docker Compose

File: `docker-compose.yml`

- Services: backend, frontend
- Ports: 8080 (backend), 80 (frontend)
- Volumes: db-data for database persistence
- Environment: Set JWT_SECRET

Nginx

File: `/etc/nginx/sites-available/mksafenet`

- HTTP to HTTPS redirect
- SSL certificate configuration
- API proxy to backend
- SPA fallback to index.html
- Static asset caching

13.3 Port Mappings

Service	Port	URL	Purpose
Backend	8080	http://localhost:8080	REST API
Frontend Dev	5173	http://localhost:5173	Vue dev server
Nginx	80/443	http/https://domain	Production proxy
Database	-	sqlite:mksafenet.db	SQLite (file-based)

Changing Ports

Backend:

```
# application.properties
server.port=8081
```

Frontend:

```
// vite.config.js
server: {
  port: 5174
}
```

Nginx:

```
server {
    listen 8080;  # Change from 80
}
```

Appendix: Additional Resources

Learning Resources

- **Spring Boot**: https://spring.io/projects/spring-boot
- **Vue.js**: https://vuejs.org/guide/
- **Pinia**: https://pinia.vuejs.org/
- **JWT**: https://jwt.io/
- **SQLite**: https://www.sqlite.org/docs.html
- **Vite**: https://vitejs.dev/

Security Resources

- **OWASP Authentication**: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- **Spring Security**: https://spring.io/projects/spring-security
- **Let's Encrypt**: https://letsencrypt.org/

Deployment Resources

- **Docker**: https://www.docker.com/
- **Nginx**: https://nginx.org/
- **Heroku**: https://www.heroku.com/
- **AWS**: https://aws.amazon.com/
- **Azure**: https://azure.microsoft.com/

Document Information

Property	Value
Document Title	MkSafeNet_Kids - Complete Project Documentation
Version	1.0.0
Date Created	June 9, 2026
Last Updated	June 9, 2026
Format	Markdown (All-in-One)
Total Sections	13 major sections
Total Pages	80+ (if printed)
Audience	Developers, DevOps, QA, Project Managers
Status	Complete & Production-Ready
